

Drawbacks of PyZenity

Even though PyZenity is very fast and simple, there are a limited number of GUI forms that you can make using this module. You cannot modify the basic GUI according to your needs. If you need two buttons in the info box, you can't have it with this module—which provides you with an info box with one button only. And you cannot modify that info box since it is a pre-defined GUI. So, PyZenity is good, but to a limited extent.

Create GUIs using Tkinter

Tkinter is my favourite among all the tool-kits. It's like creating the GUI from scratch. It takes some time to learn to use it, but believe me, it is worth spending every second of that time. It is the most portable GUI tool-kit for Python. It is also known as the top, thin, object-oriented layer of Tcl/Tk.

Get started with this module. The Tkinter tool-kit comes with the Python interpreter built in, so you don't need to manually download and install it. To use Tkinter, you have to *import* the classes from this package by using *from Tkinter import ** at the top of your program. Let's write a 'hello world' GUI with it (*demo3.py*); the output is as shown in Figure 4):

```
from Tkinter import *
root=Tk() #1
root.title("demo") #2
l=Label(root, text="Hello World", width=20) #3
l.pack() #4
root.mainloop() #5
```

Let's understand, one line at a time, what has happened above:

1. We have created a *Tk* root widget (it must be created in almost every program). It acts as a parent to all the widgets, and is a simple window with a title bar.
2. We set the title text of the window here.
3. We created a *label* using the *Label* class, with *root* as its parent widget (containing it).
4. We call the *pack* method on this widget, which tells it to size itself to fit the given text, and make itself visible.
5. We entered the Tkinter main loop. The program will stay in the event loop until you close the window. The application window will appear only after you enter the main loop.

Tkinter offers many widget classes, such as *Button*, *Label*, *Entry*, *Checkbox*, *Bitmap* (for .xpm image), etc., which let you create GUIs as per your need. These classes have many functions available to modify the style of your widget. Some of the important functions are *grid()*, *pack()*, *configure()*; each function contains a lot of parameters/arguments, like *width*, *height*, *sticky*, etc. Refer to the Web for more functions and parameters. We can also bind an event to a particular widget, if we want to. Let's create a dialogue box with the help of the 'class' method, which

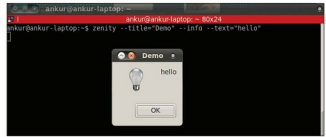


Figure 1: Output GUI

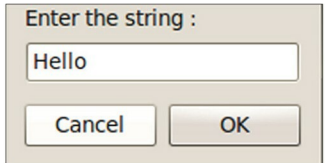


Figure 2: Output of the demo1.py

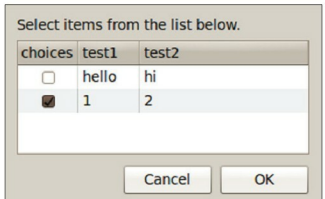


Figure 3: Output of the demo2.py

accepts the first and last name from the user. This program will demonstrate many useful aspects of Tkinter (*demo4.py*); the output is as shown in Figure 5):

```
from Tkinter import *
class diag:
    def __init__(self, parent): #1
        self.parent=parent
        self.parent.bind("<Return>", self.ok) #2
        self.parent.bind("<Escape>", self.quit) #2

    self.l1=Label(self.parent, text="First")
    self.l1.grid(row=0, sticky=W) #3
    self.l2=Label(self.parent, text="Second")
    self.l2.grid(row=1, sticky=W) #3

    self.e1=Entry(self.parent) #4
    self.e1.grid(row=0, column=1)
```